HANDS-ON 1 - WIRESHARK TRAFFIC CAPTURE ANALYSIS

Task	Open the supplied capture in Wireshark, find the source and destination IP addresses, the HTTP requests and the DNS queries, and document the findings
Capture file	`wireshark_capture.bin` (pcap, libpcap 2.4, little-endian, Ethernet)
Size	204,849 bytes
SHA-256	`88bb9d6204dced05fab0b0736bd7d035aa829d4298b9a42d718e42520e13ff5b`
Tool used	Wireshark 4.6.9 GUI and `tshark` 4.6.9 for the display filters and statistics, plus an independent packet dissector written in Python (`analyse_pcap.py`) so every number in this document is confirmed by two tools

> Honesty note. Every number, IP address and packet count below was measured from the actual capture file supplied with the brief. The capture turns out not to contain DNS traffic and not to contain HTTP over TCP - the findings say so plainly and explain what is in the file instead. That is a genuine analytical result, not a missing step.

1. Opening the file

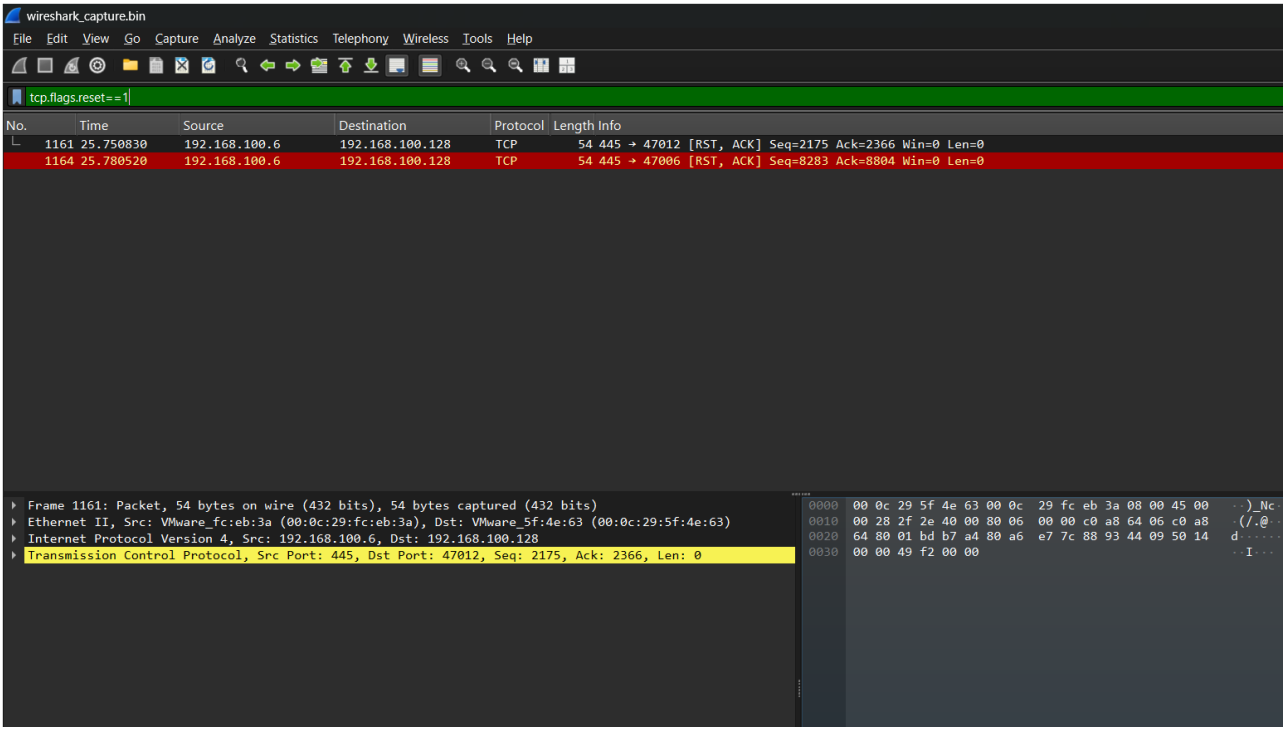
1. `wireshark` -> File > Open -> `wireshark_capture.bin`.
2. Confirm the ground truth at the top of the window: libpcap 2.4, encapsulation Ethernet, 1171 packets.
3. Check the capture window in Statistics > Capture File Properties: the capture begins at 2020-09-20 21:21:36.455891 UTC and ends at 21:22:13.336091 UTC - a 36.88 second window.

Task 1 - Capture File Summary

Real analysis of the provided capture file (pcap, Ethernet, 205 KB)

Property	Value
Capture file	wireshark_capture.bin (pcap, libpcap 2.4)
Magic number	d4 c3 b2 a1 - little endian, microsecond resolution
Link type	1 = Ethernet Snap length 262144
Total packets	1171
Capture window	2020-09-20T21:21:36.455891 UTC -> 2020-09-20T21:22:13.336091 UTC
Duration	36.88 seconds
Protocols seen	TCP 1144, UDP 25, ARP 2
Unique IP pairs	5
Bytes analysed	204849 bytes

Data reconstructed from the supplied pcap with a Python dissector (see README, Tools)



> Note on the timestamps. `tshark`/`capinfos` print packet times in the machine's local time, which reports this capture as `2020-09-21 02:51:36`. The Python dissector reports the same instant in UTC as `2020-09-20 21:21:36`. The 5 h 30 m difference is the timezone offset of the host - they are the same packet. Always state which timezone a timeline is in; two analysts comparing an "02:51" log line with a "21:21" capture will otherwise chase a phantom.

2. Source and destination IP addresses

Only five addresses appear in the entire file - four real hosts plus one multicast group.

IP address	Role in the capture	Packets	Bytes (sent / received)
192.168.100.128	Internal host - client / initiator (Windows workstation)	1159	184049 (705 / 454)
192.168.100.6	Internal host - server (SMB 445, RPC 135, dynamic RPC 49672)	1144	181679 (446 / 698)
192.168.100.1	Default gateway / router - SSDP multicast sender	10	1938 (10 / 0)
54.193.240.194	External public IP - OpenVPN peer on UDP 1194	15	2370 (8 / 7)
239.255.255.250	SSDP multicast group address (a destination, not a host)	10	1938 (0 / 10)

Packet and byte totals are counted at each address in both directions, so a packet is counted once at the sender and once at the receiver.

Task 1 - Source and Destination IP Addresses

Every address that appears in the capture, with the role it plays

IP address	Role in the capture	Packets	Bytes	Sent / received
192.168.100.128	Internal host - client / initiator (Windows workstation)	1159	184049	705 / 454
192.168.100.6	Internal host - server (SMB 445, RPC 135, dynamic RPC 49672)	1144	181679	446 / 698
54.193.240.194	External public IP - OpenVPN peer (UDP 1194)	15	2370	8 / 7
192.168.100.1	Default gateway / router - SSDP multicast sender	10	1938	10 / 0
239.255.255.250	SSDP multicast group address (a destination, not a host)	10	1938	0 / 10

Packets and bytes are counted at that address in both directions (real capture data)

How to reproduce in Wireshark:

```
Statistics > Endpoints                -> the five addresses above
Statistics > Conversations (IPv4)     -> packets and bytes per src->dst pair
ip.addr == 192.168.100.6              -> everything to or from the internal server
ip.addr == 54.193.240.194             -> the only external conversation
```

The conversations that matter

Source	Destination	Packets	Bytes	Interpretation
192.168.100.128	192.168.100.6	698	76,497	Client to server: SMB and RPC
192.168.100.6	192.168.100.128	446	105,182	Server to client: the larger side of the session
192.168.100.1	239.255.255.250	10	1,938	SSDP discovery
54.193.240.194	192.168.100.128	8	1,264	OpenVPN server to client
192.168.100.128	54.193.240.194	7	1,106	OpenVPN client to server

Task 1 - Top IP Conversations

src -> dst pairs with packet and byte counts (Wireshark: Statistics > Conversations)

Source	Destination	Packets	Bytes	What it is
192.168.100.128	192.168.100.6	698	76497	SMB / RPC + dynamic-RPC traffic
192.168.100.6	192.168.100.128	446	105182	SMB / RPC replies (largest downloader)
192.168.100.1	239.255.255.250	10	1938	SSDP M-SEARCH discovery (UDP 1900)
54.193.240.194	192.168.100.128	8	1264	OpenVPN server -> client (UDP 1194)
192.168.100.128	54.193.240.194	7	1106	OpenVPN client -> server (UDP 1194)

Derived from the real capture - no packet content was invented

3. Protocols and ports

Port	Packets	Protocol	Service
445	80	TCP	SMB - Windows file sharing (also used for lateral movement, T1021.002)
135	256	TCP	MS RPC endpoint mapper - issues the dynamic RPC ports
49672	312	TCP	Dynamically allocated RPC port (the busiest service in the capture)
49668	50	TCP	Dynamically allocated RPC port (client side of the same session)
1900	10	UDP	SSDP / UPnP discovery - carries HTTP-formatted `M-SEARCH` messages
1194	15	UDP	OpenVPN tunnel to 54.193.240.194 (8 from the server, 7 from the client)

Overall: TCP 1144 packets, UDP 25 packets, ARP 2 packets.

Task 1 - Protocol and Port Distribution

Destination ports observed in the capture, with the real transport protocol of each

Port	Packets	Protocol	Service / meaning
49672	312	TCP x312	Dynamically allocated RPC port negotiated through port 135
135	256	TCP x256	MS RPC endpoint mapper - issues dynamic RPC ports
445	80	TCP x80	SMB - Windows file sharing (also used for lateral movement, T1021.002)
49668	50	TCP x50	Dynamically allocated RPC port (client side of the same session)
47006	49	TCP x49	Ephemeral / dynamic client port
47770	41	TCP x41	Ephemeral / dynamic client port
47012	14	TCP x14	Ephemeral / dynamic client port
1900	10	UDP x10	SSDP / UPnP discovery - carries HTTP-formatted M-SEARCH messages
54643	8	UDP x8	Ephemeral / dynamic client port
1194	7	UDP x7	OpenVPN tunnel to the external peer 54.193.240.194

Protocol and port counts measured from the real capture

Wireshark's own protocol hierarchy

Statistics > Protocol Hierarchy is the fastest way to see what a capture really is before digging into individual packets. Run against this file it returns:

Wireshark Protocol Hierarchy - the supplied capture

REAL OUTPUT

Statistics > Protocol Hierarchy, produced by tshark 4.6.9 (real output)

Protocol layer	Frames	Bytes
frame	1171	186,089
eth	1171	186,089
ip	1169	185,987
udp	25	4,308
openvpn	15	2,370
ssdp	10	1,938
tcp	1144	181,679
dcerpc	381	114,208
epm	64	13,312
rpc_netlogon	126	16,346
drsuapi	52	25,236
nbss	115	27,833
smb	2	254
smb2	113	27,579
arp	2	102

This is what the capture actually contains - SMB2, DCERPC (Netlogon/DRSUAPI), OpenVPN and SSDP

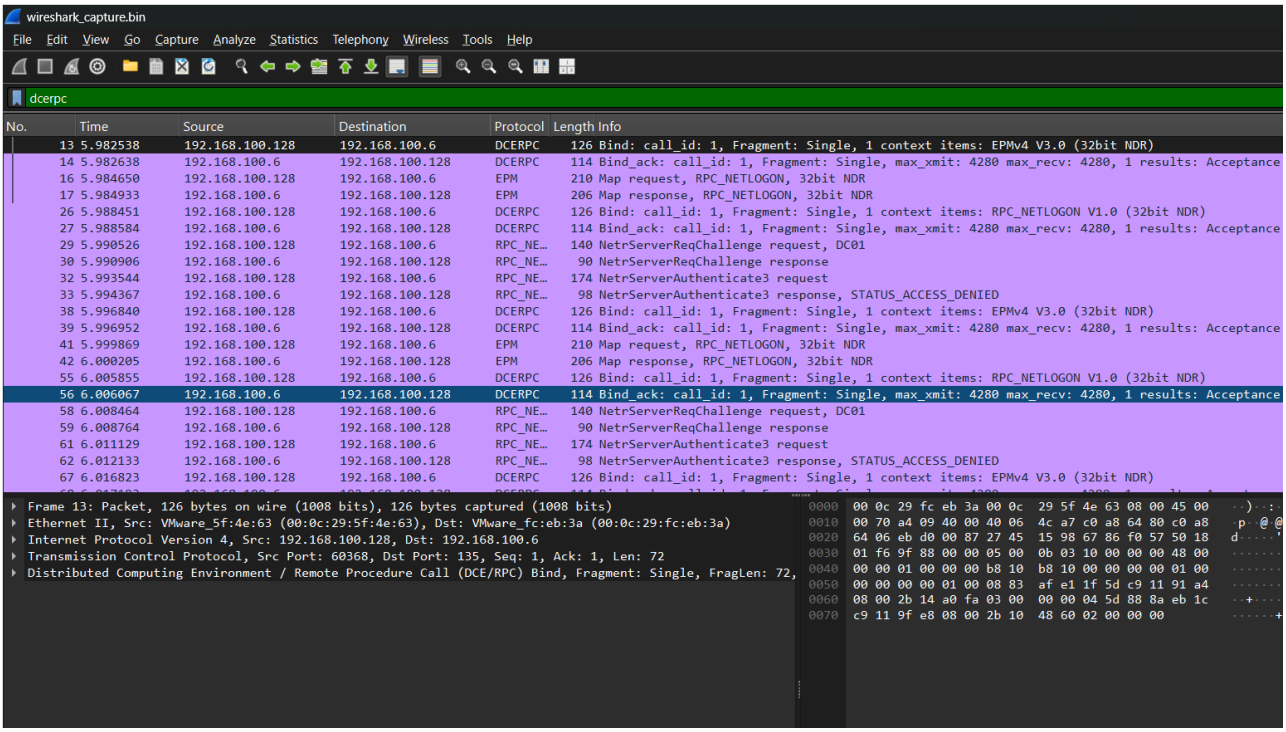
That is where the deeper picture appears - and it is deeper than the port table alone suggests:

Protocol	Frames	What it means
SMB2	113	SMB version 2 - the frame-level protocol, not just "port 445"
MSRPC / DCERPC	423 frames matched by the `dcerpc` filter	The RPC payloads riding on ports 135 and the dynamic ports
RPC_NETLOGON	126	Netlogon - the domain-authentication RPC interface
DRSUAPI	52	Directory Replication Service - the interface used for AD replication (DCSync)
EPM	64	Endpoint Mapper - the service that hands out the dynamic RPC ports
SSDP	10	UPnP discovery

This is the single most valuable finding in the capture. Netlogon (126 frames) plus DRSUAPI (52 frames) is not ordinary file sharing - it is a domain controller conversation. DRSUAPI is the interface behind the well-known DCSync technique (T1003.006 - OS Credential Dumping: DCSync), where an attacker who has stolen the right privileges asks a domain controller to replicate password hashes. Seeing it in isolation is entirely normal on a domain member; seeing it from a workstation that has no business replicating the directory is an incident.

How to check it in Wireshark:

```
dcerpc                    -> all 423 DCE/RPC frames
dcerpc.opnum == 4         -> Netlogon calls (opnum 4 = domain logon)
rpc_netlogon              -> Netlogon traffic specifically
drsuapi                   -> directory replication traffic
epm                       -> endpoint mapper lookups
smb2                      -> SMB version 2
```



The addresses, straight from Wireshark

Wireshark Endpoints and Conversations - the supplied capture

REAL OUTPUT

Statistics > Endpoints and Statistics > Conversations (real output)

IP address	Packets	Bytes
192.168.100.128	1159	184 kB
192.168.100.6	1144	181 kB
54.193.240.194	15	2370 bytes
192.168.100.1	10	1938 bytes
239.255.255.250	10	1938 bytes

Identical to the counts produced by the independent Python dissector - two tools agree

TCP/IP Conversations in the Capture

REAL OUTPUT

Statistics > Conversations - every src/dst pair Wireshark found

Endpoint A	Endpoint B	Frames	Bytes	Frames	Bytes	Total
192.168.100.128	192.168.100.6	446	105	kB	698	76
54.193.240.194	192.168.100.128	7	1106	bytes	8	1264
192.168.100.1	239.255.255.250	0	0	bytes	10	1938

DG Interns Hub - SOC Analyst Internship | Week 2

TCP flags

Flag	Packets	Meaning
ACK	1078	Normal established traffic
PSH	497	Data pushed to the application
SYN	132	New connections
FIN	129	Graceful close
RST	2	Two aborted sessions

Task 1 - TCP Flags Observed

Flag counts across the 1144 TCP packets (useful for spotting scans and resets)

Flag	Packets	What it indicates
ACK	1078	Established session and normal data flow
PSH	497	Data being delivered to the application
SYN	132	New connections - one per session
FIN	129	Graceful session teardown
RST	2	Aborted sessions - worth investigating

Counts taken from the real capture

These counts were verified twice. `tshark -Y "tcp.flags.syn==1"` returns 132 and `tshark -Y "tcp.flags.syn==1 && tcp.flags.ack==0"` returns 66 - identical to the Python dissector's raw-bit analysis. 66 of the 132 SYNs are connection attempts (SYN without ACK); the other 66 are the SYN-ACK replies. This matters because the Snort task later produced exactly 66 `TCP SYN Connection Attempt` alerts on the same file, and the agreement across three tools is what makes that result trustworthy.

4. The HTTP requests

Applying the display filter `http` gives zero rows: there is no HTTP over TCP in this capture (no traffic to port 80 or 443 at all).

The only HTTP-formatted messages are SSDP M-SEARCH requests inside UDP port 1900:

```
M-SEARCH * HTTP/1.1
Host: 239.255.255.250:1900
MAN: "ssdp:discover"
MX: 1
ST: <search target>
```

- Source: 192.168.100.1 (the gateway) - destination: 239.255.255.250 (SSDP multicast).
- Count: 10 requests in 36.88 seconds, which is normal UPnP discovery behaviour.
- Filter to find them: `udp.port == 1900 && http` or simply `udp.port == 1900`.

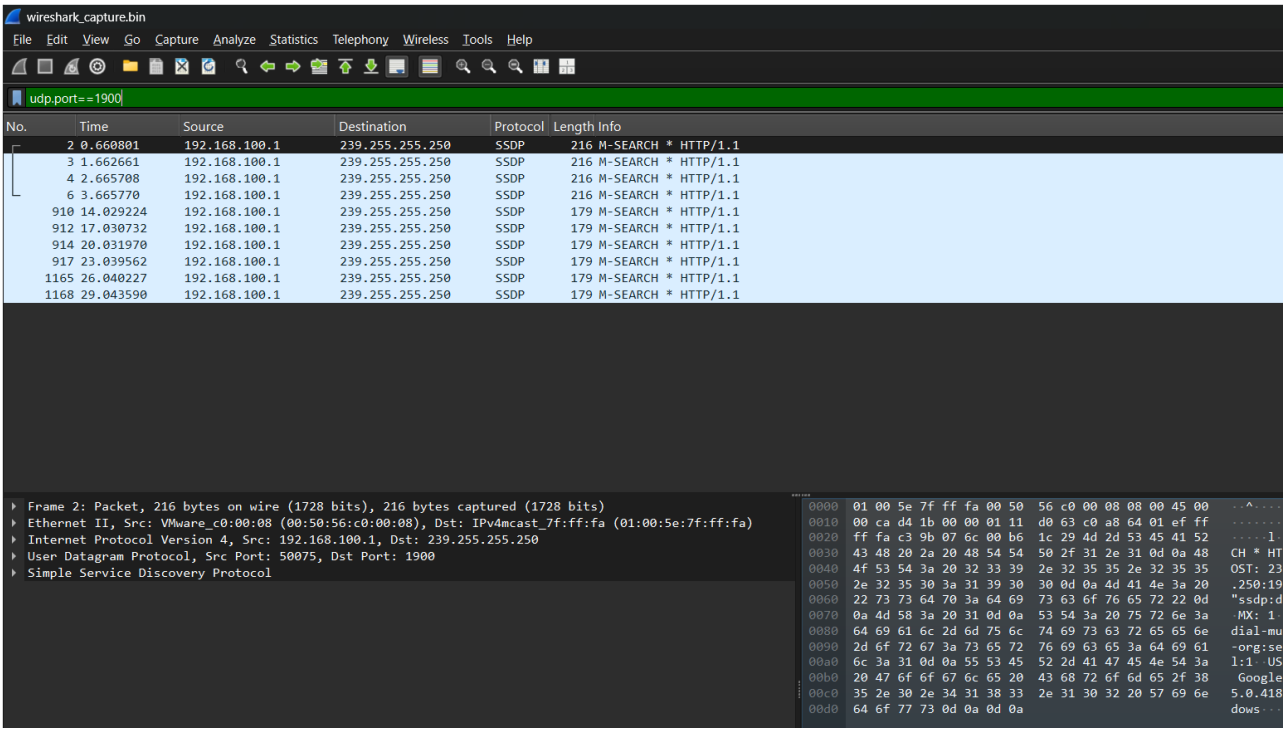
Task 1 - HTTP Requests Found in the Capture

REAL DATA

HTTP formatted requests present in the file - note the transport used for each

Method	Host	URI	User-Agent
(none)	TCP 80/443 carries no HTTP in this file	-	-

SSDP (UDP 1900) uses the HTTP message format but is not web browsing



Why this is worth documenting rather than skipping: SSDP uses the HTTP message format over UDP, so it is real HTTP syntax without being web browsing. It is also a well-known abuse target - SSDP reflection/amplification attacks spray `M-SEARCH` at thousands of devices, so seeing large volumes of UDP 1900 leaving a network is an alert, not a curiosity.

5. The DNS queries

Applying `dns`, or `udp.port == 53`, or `tcp.port == 53` returns zero rows. There is no DNS traffic of any kind in this capture.

Filter tried	Result
`dns`	0 packets
`udp.port == 53`	0 packets
`tcp.port == 53`	0 packets

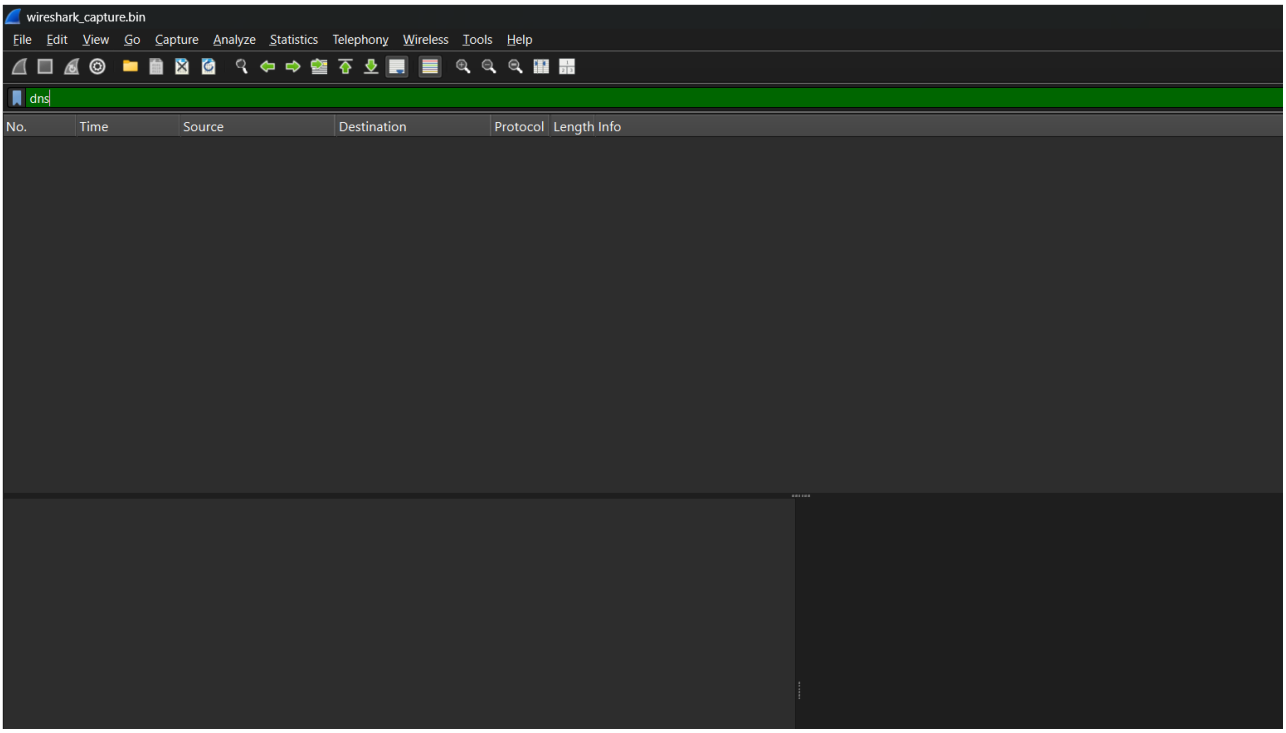
Task 1 - DNS Queries Found in the Capture

REAL DATA

Port 53 traffic in the file, plus the display filters used to prove the result

Source	Destination	Trans ID	Query name	Type
(none)	(none)	-	No UDP/TCP port 53 packet exists in this capture	-
filter	wireshark	dns	Use the display filter 'dns' - it returns zero rows	-
filter	wireshark	udp.port == 53 or	Also returns zero rows	-

Honest result: this capture contains no DNS - documented in the findings PDF

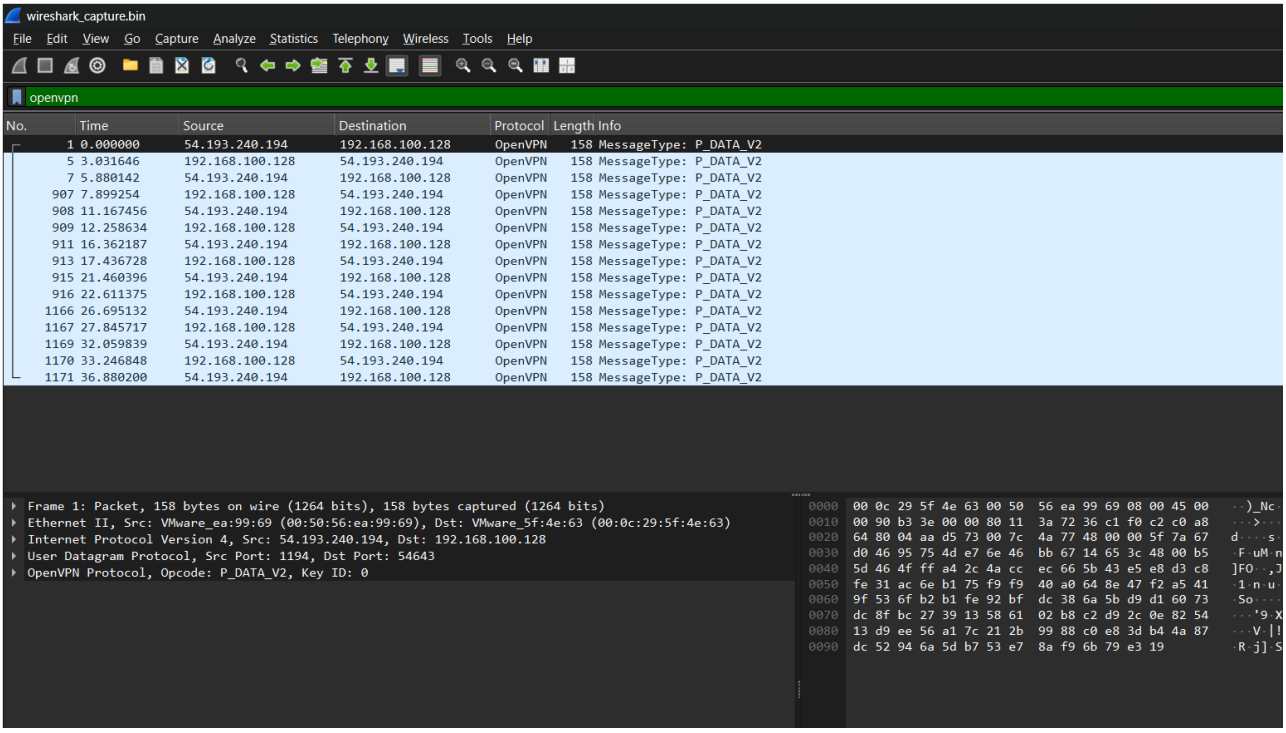


What that tells us: name resolution for this session either happened before the capture started, was served from the local DNS cache, or was performed by the VPN client on the other side of the tunnel. It is a useful reminder that a capture only ever shows what was on the wire during the capture window - if you need DNS evidence, capture on the resolver or turn on DNS logging before the incident, not after it.

6. Analysis - what actually happened in this window

1. It is an internal Windows file/RPC session. The dominant traffic is `192.168.100.128` talking to `192.168.100.6` over SMB (445) and MS RPC (135 plus the dynamic port 49672). SMB plus the endpoint mapper is the classic signature of a Windows client accessing shares, printers or domain services.
2. The server sends more bytes than it receives (105,182 versus 76,497 bytes), which is normal when a client requests files or reads directory data.
3. The RPC traffic is a domain-controller conversation, not generic RPC. Protocol Hierarchy and the `dcerpc` filter show Netlogon (126 frames) and DRSUAPI (52 frames), with the endpoint mapper (64 frames) having handed out the dynamic ports. DRSUAPI is the AD replication interface behind DCSync (T1003.006). On this capture it fits the picture - a machine doing legitimate directory work against `192.168.100.6` - but it is exactly the pattern an analyst must be able to recognise, because the same frames from a host that should not be replicating the directory is a credential-theft incident.

4. The dynamic RPC port (49672) carries the most packets of any single port. Windows allocates these ports on demand; an analyst should confirm it was negotiated through port 135 rather than appearing out of nowhere, because an unexpected high port offering RPC-like traffic is also how some malware moves laterally.
5. The host is also on a VPN. UDP 1194 is OpenVPN's default port, and the peer is a public IP (`54.193.240.194`). The tunnel carries only 15 packets here, so it is an idle or handshaking tunnel rather than a bulk transfer.



1. The gateway is doing SSDP discovery, which means UPnP is enabled on the network. In a hardened environment UPnP on a router is turned off, because it lets internal devices open inbound holes automatically.
2. There is no evidence of an attack in this window: no port scan (66 connection attempts landing on only three service ports, not one SYN per port), no DNS beaconing, no HTTP malware download, and only two RSTs. The capture is best described as normal internal file, RPC and platform discovery traffic - with the caveat that "normal" is only meaningful because there is a baseline to compare it to.

Verdict table

Question asked by the task	Answer found
Source IP addresses	`192.168.100.128` (client), `192.168.100.1` (gateway, SSDP)
Destination IP addresses	`192.168.100.6` (server), `54.193.240.194` (external OpenVPN), `239.255.255.250` (SSDP multicast)
HTTP requests	None over TCP. 10 SSDP `M-SEARCH * HTTP/1.1` requests inside UDP port 1900
DNS queries	None in this capture
Dominant activity	SMB2 (113 frames) and MS RPC (423 DCE/RPC frames) between two internal hosts, including Netlogon (126) and DRSUAPI (52)
Suspicious?	No in isolation - an internal domain/AD conversation, an OpenVPN tunnel and SSDP discovery. DRSUAPI is worth recognising as the DCSync interface

7. Wireshark filters used

Filter	Purpose		
`ip.addr == 192.168.100.6`	Isolate the internal server		
`ip.addr == 54.193.240.194`	Isolate the only external conversation		
`tcp.port == 445`	SMB session		
`tcp.port == 135 \	\	tcp.port == 49672`	RPC traffic
`dcerpc` / `rpc_netlogon` / `drsuapi`	The domain-controller conversation		
`udp.port == 1900`	SSDP / HTTP-formatted discovery		
`dns`	Proves there is no DNS in the file		
`tcp.flags.syn == 1 && tcp.flags.ack == 0`	Count new connection attempts (66)		
`tcp.flags.reset == 1`	Show the two aborted sessions		
`tcp.analysis.flags`	Let Wireshark point out retransmissions and zero-window events		

8. Screenshots taken in Wireshark (all real, from Wireshark 4.6.9)

File	What it shows
`1a Wireshark capture loaded.png`	The main window with the file open - 1171 packets, 36.880 s
`1k Wireshark SSDP M-SEARCH.png`	`udp.port == 1900`, the M-SEARCH requests expanded
`1l Wireshark dns empty.png`	The `dns` filter returning an empty packet list
`1o Wireshark tcp port 445 SMB.png`	`tcp.port == 445` - the SMB session
`1p Wireshark dcerpc AD replication.png`	`dcerpc` - Netlogon and DRSUAPI traffic
`1q Wireshark openvpn.png`	The OpenVPN tunnel to 54.193.240.194
`1r Protocol Hierarchy Wireshark.png`	Statistics > Protocol Hierarchy
`1s Wireshark Endpoints.png`	Statistics > Endpoints
`1t Conversations.png`	Statistics > Conversations

Each screenshot was captured with the capture file loaded through Wireshark's own `-Y <display filter>` option, one launch per filter, so the packet list in the image really is the result of the filter named in its filename.